Simple Java applications. Java IDEs

Objectives

- Theoretical and practical understanding of the basic elements of Java programming.
- Writing simple programs, object oriented, using the Eclipse integrated environment.
- Compiling and running the applications from the command line
- Simple Java applications with input/output operations.

The Java language appeared (1996, the official launch of the first version of Java) as an alternative to the C++ software environment and it was intended to keep all the positive aspects manifested by it and to eliminate the negative ones. We can say that the Java language had C/C++ as its parent, the Java and C/C++ syntaxes being very close. It can also be said that for a programmer familiar with the C/C++ language, getting used to the Java environment should not present any kind of problem. The Java language incorporates, in addition to the essential inheritance received from C++, a multitude of aspects taken from a variety of other third-party programming languages.

The Java platform is a purely software platform.

The Java platform contains 3 components:

- The *Java Virtual Machine* (JVM)
- The *Application Programming Interface* (API)
- The *Java Runtime Environment* (JRE)

Types of Java applications

The Java applications can be classified as follows:

- ***stand-alone applications***. Their essential feature is that they contain the method *public static void main(String[] args*) that constitutes the entry point in the application. They can have different functionalities, can be run from the command line, can have graphical user interaction interfaces, etc.
- ***applets.*** They are the applications intended for running in a web browser. They have been eliminated since Java11.
- ***servlets***. A servlet runs inside a web server and communicates (I/O operations) using the HTTP protocol. Java servlets are used to build interactive Web applications, being a replacement for CGI scripts. The web applications have *frontend* and *backend* facilities that manage the user interaction, the database communication, etc.
- ***etc.***

JDK – Java Developers's Kit

Contains all the necessary tools for creating, debugging and executing Java programs.

http://www.oracle.com/technetwork/java/javase/downloads/index.html

The documentation related to Java SE21:

https://docs.oracle.com/en/java/javase/25/docs/api/index.html

Java source files: *.java

Java bytecodes: *.class

JDK – Directories structure

bin => directory for storing Java utility and executable files. Here reside *appletviewer.exe, jar.exe, java.exe, javac.exe, javadoc.exe, javah.exe, javap.exe, jdb.exe, serialver.exe, servertool.exe,* etc.

conf => directory that stores files that contain user-configurable options

- changing JDK access permissions
- security algorithms configuration
- setting the Java Cryptography Extension Policy files

include => directory for storing C/C++ header files, **.h*;

jmods => stores compiled modules, used by *jlink* for creating a personalized *runtime* environment.

legal => directory for keeping the license and copyright files for each module

lib => directory for storing Java libraries (**.jar*, **.sym*, **.lib*, **.idl*).

Java utility files

Java utilities, as mentioned earlier, are stored in the ***bin*** directory located at the JDK installation location. On JDK versions intended for Microsoft systems, Java utilities are typically executable files **.exe*. Any of them can be run from the command line. Any Java utility supports several options that the user is informed about (usually) by writing the name of the desired executable file directly in the command line.

javac.exe = compiler

javac [options] [source_files] [command_line_arguments]

Example: source file *Test.java*

Compiling variants:

javac Test.java

javac –nowarn Test.java invalidates the warning messages

*javac *.java* compiles all files with *java* extension in the current directory

java.exe = interpreter

java [options] ClassName [command_line_arguments]

java [options] -jar java_archive.jar [command_line_arguments]

Example: after a successful compiling → *Test.class*

running: *java Test*

Documentation generation /**...*/ ***javadoc*** *SourceFile.**java***

Disassembling bytecode files (.class): ***javap*** *ClassName.**class***

Java debugger: ***jdb*** *ClassName*

Java IDE

IDE = Integrated Development Environment

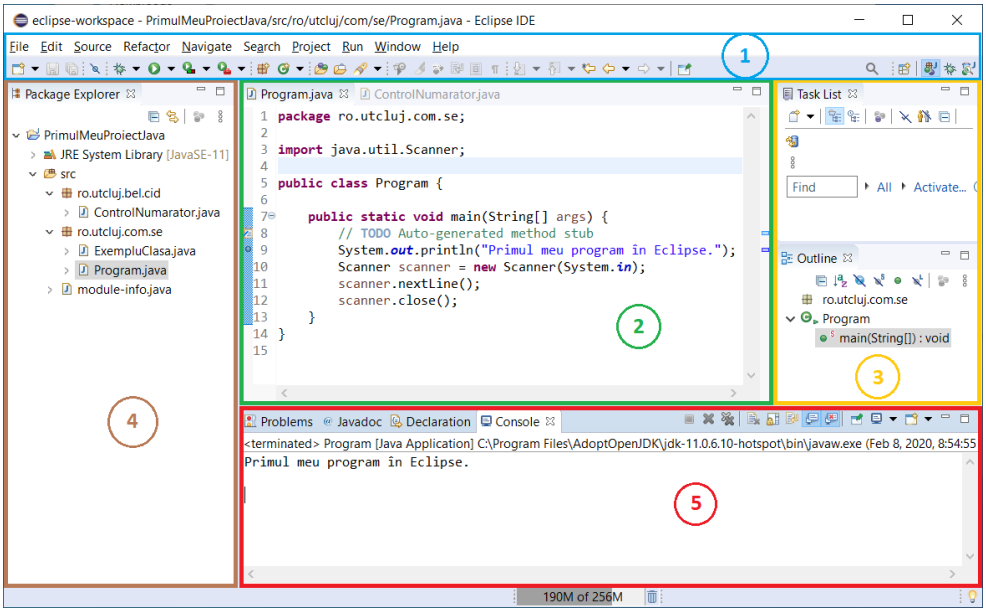
- *Eclipse,* https://www.eclipse.org/
- ApacheNetBeans, https://netbeans.apache.org/download/index.html
- IntelliJ IDEA, https://www.jetbrains.com/idea/download/
- etc.

The Eclipse Integrated Development Environment

The work perspectives are display modes for the different tasks a programmer can do. There are 2 commonly used work perspectives:

1. **The Java perspective**, used to code, to navigate the project file hierarchy and the class hierarchy, and to view work tasks. This is the default perspective in Eclipse for Java projects.
2. **The Java Debug perspective**, used to troubleshoot Java programs and execute them step by step. It opens automatically when a program is started in debug mode, and consists in a series of areas for viewing the running threads, variables, debug stopping points, and the console. Note: the console is displayed as a window of the development environment.

Java perspective



Area 1 represents the menu bar, located in the top half, and the button bar. The left side of the button bar facilitates the creation of new projects, classes, interfaces or other entities, saving documents, and is followed by a special area for launching or debugging programs in troubleshooting or running. The working perspective can be changed from the right side of the button bar.

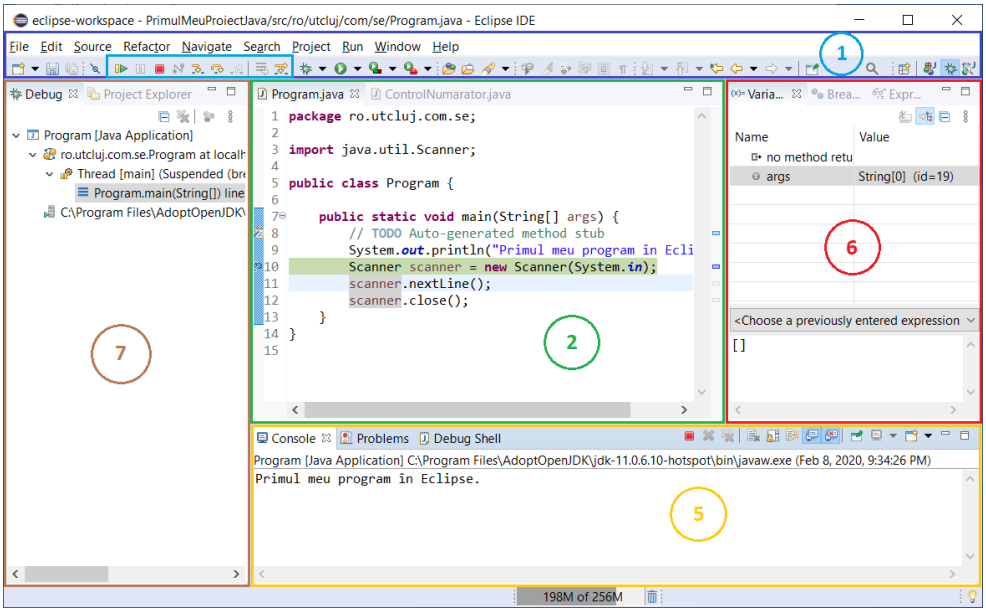
Area 2 is contains the code editor.

Area 3 represents the Outline window and the tasks list. The Outline window displays the program elements available in the code editor.

Area 4 represents the project and package explorer.

Area 5 contains windows for displaying the potential problems in the program, the documentation generated with javadoc, the line on which a variable selected in the code editor has been declared, and the console. This area is accompanied a button bar, which allows the user to do operations with the console, to delete the displayed text, to select a specific console in case several programs have been launched into execution simultaneously.

Debug perspective



Area 1 is generally the same as in the Java perspective, with a notable modification: the buttons for creating new elements of code have been replaced with buttons for controlling the step-by-step execution of the program. The step-by-step execution is useful because it allows the programmer to visualize, at a certain moment, the values that the different variables have and other relevant details.

Area 2 is the same as in the Java perspective, but in the image above line 10 has been marked by a breakpoint in the left bar. The highlight means that the execution of the program was stopped at that line for inspection.

Area 5 provides access to the console, to the window with the list of potential problems, and to the window for executing ad hoc code in the context of the program.

Area 6 displays a list of visible variables at the line at which the inspected program was stopped. Also, this area has windows for managing the stopping points, for inspecting and for evaluating various expressions in the program's context.

Area 7 displays a window with the list of programs being debugged, each broken down at thread-level execution. When a program contains several execution threads and is stopped for inspection, the user can navigate between them to inspect the appropriate condition of each thread.

Tutorials

https://helios.utcluj.ro/media/InstalareJDK_Eclipse.mp4

https://helios.utcluj.ro/media/SetareVariabileSistem.mp4

https://helios.utcluj.ro/media/Creare_Rulare_ProiectEclipse.mp4

https://helios.utcluj.ro/media/RulareLinieComanda.mp4

Simple Java applications

Hello World application

```
public class Test {

	public static void main(String args[]){
		System.out.println("\nHello Test");
	}

}
```

I/O applications

a) I/O using BufferedReader

```
//import java.io.DataInputStream;
	import java.io.*;
	import java.lang.String;
	import java.io.IOException;

	public class LeapYear {
		public static void main(String args[]){
			// DataInputStream dis = new DataInputStream(System.in);
		BufferedReader dis= new BufferedReader(new InputStreamReader(System.in));
			String Syear=null;
			System.out.print("Enter a year: ");
			System.out.flush();
			try {
			  Syear = dis.readLine();
			} catch (IOException ioe){
				System.out.println(ioe.toString());
				System.exit(1);
			  }

			long year = Long.parseLong(Syear);
```

```
                System.out.println("year is " + year);
                if ( ((year % 4 == 0) && (year % 100 != 0))  ||
                      (year % 400 == 0) )
                        System.out.println(year + " is a leap year!");
                else
                        System.out.println(year + " is NOT a leap year.");
            }
        }
```

b) I/O using Scanner

```
//import java.io.DataInputStream;
        import java.util.Scanner;

//      import java.lang.String;
//      import java.io.IOException;

        public class LeapYear {
                private static Scanner dis;
                 public static void main(String args[]){
                     //DataInputStream dis = new DataInputStream(System.in);
                      dis = new Scanner(System.in);//System.in is an InputStream

                        String Syear=null;
                        System.out.print("Enter a year: ");
                        System.out.flush();
                       // try {
                         // Syear = dis.readLine();
                            Syear = dis.next();
                        //} catch (IOException ioe)
                         // {
                        //       System.out.println(ioe.toString());
                        //        System.exit(1);
                        //  }

                        long year = Long.parseLong(Syear);
                        System.out.println("year is " + year);
                        if ( ((year % 4 == 0) && (year % 100 != 0))  ||
                              (year % 400 == 0) )
                                System.out.println(year + " is a leap year!");
                        else
                                System.out.println(year + " is NOT a leap year.");
                 }
        }



//Leap year clean version
        import java.util.Scanner;

        public class LeapYear {
                private static Scanner dis;
                 public static void main(String args[]){
                             dis = new Scanner(System.in);//System.in is an InputStream
```

```
                long year;
                System.out.print("Enter a year: ");
                System.out.flush();
                year = dis.nextLong();
                System.out.println("year is " + year);
                if ( ((year % 4 == 0) && (year % 100 != 0))  ||
                      (year % 400 == 0) )
                        System.out.println(year + " is a leap year!");
                else
                        System.out.println(year + " is NOT a leap year.");
            }
        }
```

c) I/O using Console (only from command prompt !)

```
//import java.io.DataInputStream;
      import java.io.Console;

      //import java.lang.String;
    //import java.io.IOException;

      public class LeapYear {
            //private static Scanner dis;
             public static void main(String args[]){
                 //DataInputStream dis = new DataInputStream(System.in);
                  Console dis = System.console( );

                   String Syear=null;
                System.out.print("Enter a year: ");
                System.out.flush();
                Syear = dis.readLine();

                long year = Long.parseLong(Syear);
                System.out.println("year is " + year);
                if ( ((year % 4 == 0) && (year % 100 != 0))  ||
                      (year % 400 == 0) )
                        System.out.println(year + " is a leap year!");
                else
                        System.out.println(year + " is NOT a leap year.");
            }
      }
```

C:\WINDOWS\system32\cmd.exe

```
Microsoft Windows [Version 10.0.19042.804]
(c) 2020 Microsoft Corporation. All rights reserved.

e:\Java_2021\Test\bin>java LeapYear
Enter a year: 2021
year is 2021
2021 is NOT a leap year.

e:\Java_2021\Test\bin>
```

d) I/O using DataInput.Stream (deprecated)

```
import java.io.DataInputStream;
import java.lang.String;
import java.io.IOException;

public class LeapYear {
        @SuppressWarnings("deprecation")
         public static void main(String args[]){
                DataInputStream dis = new DataInputStream(System.in);
                String Syear=null;
                System.out.print("Enter a year: ");
                System.out.flush();
                try {
                  Syear = dis.readLine();
                } catch (IOException ioe)
                  {
                        System.out.println(ioe.toString());
                        System.exit(1);
                  }

                long year = Long.parseLong(Syear);
                System.out.println("year is " + year);
                if ( ((year % 4 == 0) && (year % 100 != 0))  ||
                      (year % 400 == 0) )
                        System.out.println(year + " is a leap year!");
                else
                        System.out.println(year + " is NOT a leap year.");
         }
}
```

Simple operations using the Math library

```
import java.lang.Math;

public class MathM{
       public static void main(String[] args){
       double x,y,z;
       int i;
              i=10;
              y=Math.log(10.0);
              x=Math.pow(3.0,4.0);
              z=Math.random(); // random number from 0 to 1
              System.out.println("Value of i is " + i);
              System.out.println("Value of log(10) is " + y);
              System.out.println("Value of 3 at power 4 is " + x);
              System.out.println("A random number is " + z);
              System.out.println("Square root of 2 is " +   Math.sqrt(2));
       }
}
```

Graphical application

```
import java.awt.Color;
import java.awt.Graphics;
import javax.swing.JFrame;
import javax.swing.JPanel;

//Clasa care deriva din JPanel pentru a putea modifica modul
//in care panoul este desenat pe ecran

class PanouExemplu extends JPanel {
    private static final long serialVersionUID = 1L;
    //metoda desenare mostenita din clasa de baza
    public void paintComponent(Graphics g) {
        int totalP=6;
        int xP1[]={110,50,200,20,170,110};
        int yP1[]={20,190,80,80,190,20};
        g.drawPolygon(xP1,yP1,totalP);
        int xP2[]={310,250,400,220,370,310};
        int yP2[]={20,190,80,80,190,20};
        g.setColor(Color.blue);
        g.fillPolygon(xP2,yP2,totalP);
      }
    }//PanouExemplu

public class JPanelTest{
       public static void main(String args[ ]) {
        JFrame frame = new JFrame("Exemplu desenare pe panou.");
        frame.add(new PanouExemplu());
        frame.setSize(440, 300);
        frame.setVisible(true);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }//main
}
```

Proposed activities

Complexity *

1. Verify the functionalities of the Eclipse environment considering the applications given as examples.
2. Write a Java application where you read from the keyboard an *int* value. Calculate and display the factorial of the read number.
3. Write a Java application that reads from the keyboard 2 integer values and displays their greatest common divider (GCD).
4. Write a Java application where you read from the keyboard an integer value and display all its dividers.
5. Write a Java application where you read several integers from the keypad and verify if they are prime numbers. The number of values is also read from the KB (or command line).